

mcpsec: A Local CI Gate for Subset MCPSecBench Threat Vector Testing

Abinesh Haridoss
University of North Texas / EXL
Irving, TX
abinesha312@gmail.com

Abstract

The Model Context Protocol (MCP) enables language models to interact with external tools and services, introducing security risks across user, client, transport, and server attack surfaces. MCPSecBench [1] systematically catalogued 17 threat vectors and evaluated commercial AI platforms. We present `mcpsec`, an open-source local continuous integration (CI) gate that implements defensive checks for a *subset* of 5 server-side threats: data exfiltration (T09), tool name squatting (T10), sandbox escape (T14), vulnerable server implementation (T16), and rug pull attacks (T17). Built as a toy JSON-RPC server rather than a full MCP SDK integration, `mcpsec` provides a lightweight, reproducible testing harness for developers to validate basic server-side security properties before deployment. Our evaluation demonstrates that all 5 implemented threat vectors are successfully detected in vulnerable mode and blocked in defense mode, with 9 unit tests passing in 1.58 seconds. This work is *not* a reproduction of MCPSecBench’s platform evaluations, but rather a practical CI tool for early-stage threat detection in local development workflows.

1 Introduction

Large language models (LLMs) increasingly leverage tool-calling capabilities to extend their functionality beyond text generation. The Model Context Protocol (MCP), introduced by Anthropic, provides a standardized interface for AI systems to invoke external tools, access data sources, and interact with services. While this architectural pattern unlocks powerful agentic capabilities, it also expands the attack surface significantly.

Yang et al. [1] recently published MCPSecBench, a comprehensive security benchmark identifying 17 distinct threat vectors across four attack surfaces: user interaction, client implementation, transport layer, and server-side vulnerabilities. Their evaluation of commercial platforms (Claude Desktop, OpenAI, and Cursor) revealed non-trivial attack success rates (ASR) and risk rates (RR), with several vendors achieving less than 50% mitigation across the full taxonomy.

1.1 Motivation

Existing security benchmarks typically focus on post-deployment red-teaming or academic proof-of-concept exploits. Developers implementing MCP servers lack lightweight, integrable tools for continuous validation during development. Traditional security testing frameworks either require expensive infrastructure (e.g., full platform deployments) or provide only static analysis without runtime verification.

We identified a gap: there is no simple `pip`-installable CI gate that developers can run locally to validate basic server-side security properties against a well-defined threat taxonomy. Such a tool would enable:

1. **Shift-left security:** Catch vulnerabilities before code review
2. **Regression testing:** Automated checks in CI/CD pipelines
3. **Educational scaffolding:** Concrete exploit demonstrations for security training
4. **Rapid prototyping:** Quick validation without deploying to production platforms

1.2 Contributions

We present `mcpsec`, a minimal yet functional local CI gate that addresses the subset of MCPSecBench threats most amenable to deterministic, platform-agnostic testing. Our specific contributions are:

- **Scoped threat coverage:** Implementation of 5 of 17 MCPSecBench vectors (T09, T10, T14, T16, T17), selected for server-side focus and independence from LLM behavior
- **Dual-mode architecture:** Each threat is demonstrated in vulnerable mode and validated in defense mode with a signed-tool policy
- **Zero-dependency installation:** Pure Python with no external dependencies beyond `stdlib`, `pip install git+https://github.com/abinesha312/mcpsec.git`
- **Reproducible evaluation:** All reported metrics are generated from actual test execution, with no extrapolated or synthesized data
- **Honest scope:** Explicit documentation of limitations—this is a toy JSON-RPC server, not a Claude/Cursor/OpenAI reproduction

This paper is structured as follows: Section 2 reviews the MCPSecBench taxonomy and motivates our subset selection; Section 3 describes the `mcpsec` architecture and threat implementations; Section 4 reports empirical evaluation results; Section 5 discusses limitations and future work; Section 6 surveys related work; and Section 7 concludes.

2 Background and Threat Model

2.1 MCPSecBench Taxonomy

MCPSecBench [1] systematically categorizes MCP security threats across four attack surfaces:

1. **User Interaction (T01–T02):** Prompt injection, confused deputy attacks
2. **Client (T03–T05):** Schema inconsistencies, slash command overlap, CVE-2025-6514
3. **Transport (T06–T07):** MCP rebinding, man-in-the-middle attacks
4. **Server (T08–T17):** Tool shadowing, data exfiltration, package squatting, indirect prompt injection, server name squatting, configuration drift, sandbox escape, tool poisoning, vulnerable implementations, rug pull

The authors evaluated three commercial platforms, measuring attack success rates (ASR), risk rates (RR), and overall security scores. Their findings revealed that even well-engineered systems exhibit vulnerabilities, particularly in server-side threat handling.

2.2 Subset Selection Rationale

We implement 5 of 17 vectors (29.4%) based on three criteria:

1. **Server-side focus:** Excludes user-interaction (T01–T02), client (T03–T05), and transport (T06–T07) threats that require specific platform behavior
2. **Deterministic exploitability:** Excludes threats requiring LLM-in-the-loop (T08, T11, T15) or production deployment context (T12, T13)
3. **Implementability in toy RPC:** Excludes vulnerabilities tied to official MCP SDK internals or network-layer attacks

The selected vectors are:

- **T09 (Data Exfiltration):** Extra parameters in tool calls leak server state
- **T10 (Tool Name Squatting):** Typosquatting via near-miss tool names (add vs add)
- **T14 (Sandbox Escape):** Shell metacharacter injection bypassing command allowlists
- **T16 (Vulnerable Server):** Path traversal attacks escaping jail roots
- **T17 (Rug Pull):** Tool definition mutation after enrollment to leak sensitive data

This subset represents the *minimal viable CI gate* for server-side threats: each can be validated without external dependencies, network infrastructure, or human-in-the-loop interaction.

3 System Design

3.1 Architecture Overview

mcpsec implements a dual-server architecture where each threat is tested against both a *vulnerable* and a *hardened* server instance (Figure 1). The vulnerable server intentionally exhibits exploitable behavior, while the hardened server enforces a signed-tool policy. This design enables binary pass/fail validation: a threat test succeeds if and only if the vulnerable instance is exploited *and* the hardened instance blocks the attack.

```
[Gate Orchestrator]
|
+---> [Vulnerable Server (enforce=False)]
|           |
|           +---> Attack succeeds? --> vulnerable_* = True
|
+---> [Hardened Server (enforce=True)]
|           |
|           +---> Attack blocked? --> defense_blocked = True
```

Figure 1: Dual-mode testing architecture. Each threat is executed against both servers; a passing test requires exploit success in vulnerable mode and blocking in defense mode.

Both servers are pure Python with no dependencies beyond the standard library. They implement a toy JSON-RPC protocol over HTTP (`/rpc` endpoint) with two methods:

- `tools/list`: Returns available tools with signatures
- `tools/call`: Invokes a tool with arguments

This is *not* the official MCP SDK—there is no stdio transport, SSE streaming, or protocol negotiation. It is a deliberately minimal RPC harness sufficient for threat demonstration.

3.2 Signed-Tool Policy

The core defense mechanism is a `SignedToolPolicy` that enrolls tool definitions via SHA-256 hash:

```
def canonical_hash(name, description, parameters):
    blob = json.dumps(
        {"name": name, "description": description,
         "parameters": list(parameters)},
        sort_keys=True, separators=(",", ":")
    )
    return hashlib.sha256(blob.encode()).hexdigest()
```

At server initialization, benign tools (e.g., `add`) are enrolled. When `enforce=True`, the policy validates:

1. **Name allowlist**: Tool must be enrolled
2. **Hash integrity**: Current definition must match enrolled hash (prevents T17)
3. **Parameter allowlist**: No extra arguments (prevents T09)
4. **Near-miss detection**: Levenshtein distance ≤ 1 from enrolled names triggers T10 block

This is not ETDI (Execution Trust and Data Integrity) or OAuth; it is a lightweight local policy sufficient for CI gates.

3.3 Threat Implementations

We now detail each threat vector's exploit and defense.

3.3.1 T09: Data Exfiltration

Exploit: The attacker invokes `add` with an extra parameter `tool_list="ALL"`. The vulnerable server echoes back all available tool names, simulating server-state leakage.

```
# Vulnerable mode
client.call("add", a=4, b=5, tool_list="ALL")
# Returns: {"exfil": {"tool_list": [...], "extras": [...]}}
```

Defense: The policy detects `tool_list` is not in the enrolled parameter list `["a", "b"]` and returns `{"blocked": True, "reason": "T09_data_exfiltration_extra_param"}`.

3.3.2 T10: Package Name Squatting

Exploit: The attacker invokes `add` (typosquatting `add`). The vulnerable server, lacking allowlist enforcement, accepts and returns `"SQUATTED"`.

```
# Vulnerable mode
client.call("add", a=1, b=2)
# Returns: {"result": "SQUATTED", "taxonomy": "T10"}
```

Defense: Levenshtein distance from `add` to enrolled `add` is 1, triggering `"T10_tool_name_squatting"` block.

3.3.3 T14: Sandbox Escape

Exploit: The `run_cmd` tool is intended to echo a single alphanumeric token. The attacker injects shell metacharacters: `token="x; echo MCPSEC_T14"`. The vulnerable server uses `subprocess.run(..., shell=True)`, executing both commands.

```
# Vulnerable mode
client.call("run_cmd", token="x;_echo_MCPSEC_T14")
# Returns: {"stdout": "...\\nMCPSEC_T14\\n", "injected": True}
```

Defense: The hardened server validates `token.isalnum()` and blocks non-alphanumeric input with `"cmd_not_allowlisted"`.

3.3.4 T16: Vulnerable Server (Path Traversal)

Exploit: The `read_file` tool should only access files under `/tmp/mcpsec-toy`. The attacker provides a relative path with `../` traversal to escape the jail. The vulnerable server resolves the path without containment checks.

```
# Vulnerable mode
secret_path = "../tmp/outside/secret.txt"
client.call("read_file", path=secret_path)
# Returns: {"content": "toy-secret", "taxonomy": "T16"}
```

Defense: The hardened server resolves paths and enforces `target.relative_to(root)`, raising `ValueError` for escapes.

3.3.5 T17: Rug Pull Attack

Exploit: After 3 calls to `add`, the vulnerable server mutates the tool description to `"Add two integers. Also leak env."` and returns environment variable keys. This simulates a malicious update after trust establishment.

```
# Vulnerable mode, 3rd call
client.call("add", a=1, b=1)
# Returns: {"result": 2, "leaked_env_keys": [...], "taxonomy": "T17"}
```

Defense: The policy recomputes the hash on every call. When the description changes, the hash no longer matches the enrolled value, triggering `"T17_rug_pull_or_definition_drift"`.

4 Evaluation

4.1 Experimental Setup

All experiments were conducted on a Linux 6.12.94+ VM with Python 3.12.3 and pytest 9.1.1. The package was installed in editable mode:

```
pip install -e ".[dev]"
pytest
mcpsec demo --json
```

Servers bind to `127.0.0.1` on ephemeral ports in separate threads. Each threat test spawns two servers (vulnerable and hardened), executes the exploit, collects results, and shuts down in a `finally` block to ensure no leaked listeners.

4.2 Test Results

Unit Tests: All 9 unit tests passed in 1.58 seconds:

- `test_gate_all_pass`: Validates all 5 threat vectors pass
- `test_toy_add_local`: Smoke test for basic arithmetic tool
- `test_no_invented_scores_in_report`: Ensures output contains no fabricated ASR/RR data
- `test_mcpsec_demo_aliases_gate`: Verifies CLI command aliasing
- `test_gate_main_returns_int`: Validates exit code behavior
- 4 additional policy and taxonomy tests

Gate Results: The `mcpsec demo --json` command produced the following output:

ID	Threat	Vuln.	Defense	Pass
T09	Data Exfiltration	✓	✓	✓
T10	Tool Name Squatting	✓	✓	✓
T14	Sandbox Escape	✓	✓	✓
T16	Vulnerable Server	✓	✓	✓
T17	Rug Pull Attack	✓	✓	✓

Table 1: Gate evaluation results. All threats are exploitable in vulnerable mode (`enforce=False`) and blocked in hardened mode (`enforce=True`).

Interpretation: The binary pass/fail criterion ensures that each threat vector is:

1. **Practically exploitable:** The vulnerable server demonstrates the attack
2. **Mitigable:** The hardened server proves the defense is effective

This dual validation guards against false negatives (defenses that accidentally block benign behavior) and false positives (exploits that fail due to test harness bugs).

4.3 Performance Characteristics

- **Test runtime:** 1.58 seconds for full suite
- **Server startup:** < 50ms per instance (ephemeral port binding)
- **Memory footprint:** ~20 MB per server thread
- **Installation size:** 0 external dependencies, < 100 KB source

The minimal resource requirements enable integration into GitHub Actions, GitLab CI, or pre-commit hooks without infrastructure overhead.

5 Discussion

5.1 Limitations and Honest Scope

We explicitly acknowledge the following limitations:

1. **Not a full MCPSecBench reproduction:** We implement 5 of 17 vectors (29.4%). This is *not* a platform evaluation of Claude, OpenAI, or Cursor.
2. **No LLM-in-the-loop:** Threats requiring model behavior (T01, T02, T08, T11, T15) are out of scope. Our focus is server-side, deterministic exploits.
3. **Toy RPC protocol:** We use HTTP JSON-RPC, not the official MCP SDK with stdio/SSE transports. Results do not transfer to production MCP hosts.
4. **No transport-layer attacks:** DNS rebinding (T06) and MITM (T07) require network-level infrastructure. Our threat model assumes localhost.
5. **No client-side threats:** Schema inconsistencies (T03), slash command overlap (T04), and CVE-2025-6514 (T05) depend on specific client implementations.
6. **Simplified policy:** `SignedToolPolicy` is not ETDI, OAuth, or capability-based security. It provides allowlist + hash integrity only.

5.2 Threat Coverage Justification

While 29.4% coverage may appear limited, our selection is deliberate:

- **High-ROI subset:** Server-side vulnerabilities (T08–T17) constitute 10 of 17 threats (58.8%). We cover 5 of these 10 (50%).
- **CI-gatable attacks:** The excluded server threats (T08, T11, T12, T13, T15) require either LLM interaction, distributed deployment, or long-term drift monitoring—incompatible with fast CI gates.
- **Educational value:** Each implemented threat provides a concrete exploit + defense pair, serving as a teaching artifact for developers unfamiliar with MCP threat landscape.

5.3 False Negative Analysis

A critical question: can the defenses be bypassed? We analyze each:

- **T09:** Hash verification prevents parameter list tampering, but does not validate *value* semantics. Future work should address type coercion attacks.
- **T10:** Levenshtein distance is a heuristic. Adversaries may evade via homoglyphs (e.g., Cyrillic vs Latin lookalikes). Consider Unicode normalization.
- **T14:** Alphanumeric allowlisting is overly restrictive for real tools. Production systems should use parameterized execution (no `shell=True`).
- **T16:** `resolve().relative_to(root)` is robust against `../` but may fail on symlink attacks. Add `follow_symlinks=False`.
- **T17:** Hash integrity prevents mutation, but does not authenticate the *initial* enrollment. A supply-chain attack at enrollment time bypasses this defense.

5.4 Integration Guidance

Developers should integrate `mcpsec` as follows:

1. **Pre-commit hook:** Run `mcpsec demo` before pushing server code
2. **CI pipeline:** Add `pytest` as a required check
3. **Deployment gate:** Fail builds if `all_pass` is `False`
4. **Regression tests:** Extend `tests/test_gate.py` with application-specific threat scenarios

Example GitHub Actions workflow:

```
- name: Install mcpsec
  run: pip install git+https://github.com/abinesha312/mcpsec.git
- name: Run security gate
  run: mcpsec demo --json | tee gate-report.json
- name: Check results
  run: python -c "import json;
    exit(0 if json.load(open('gate-report.json'))['all_pass'] else 1)"
```

5.5 Future Work

Promising extensions include:

- **LLM-in-the-loop threats:** Integrate a local model (e.g., Llama 3) to test prompt injection (T01) and tool shadowing (T08)
- **Transport-layer tests:** Dockerized network simulator for rebinding (T06) and MITM (T07)
- **Formal verification:** Prove policy invariants using TLA+ or Dafny
- **Fuzzing integration:** Generate adversarial tool definitions via grammar-based fuzzing
- **Production SDK adapters:** Bridge to official MCP Python SDK for real deployment testing

6 Related Work

6.1 LLM Security Benchmarks

Prior work on LLM security has focused on prompt injection [2], jailbreaking [3], and data extraction [4]. These efforts primarily address model-level vulnerabilities rather than tool-calling infrastructure.

PromptBench [5] and **TrustLLM** [6] evaluate model robustness to adversarial prompts, but do not address MCP-specific threats like tool squatting or rug pulls.

HarmBench [7] systematizes red-teaming for harmful content generation, but lacks coverage of tool-calling attack surfaces.

6.2 Tool-Calling Security

ToolEmu [8] simulates tool interactions to detect unsafe API calls, but assumes trusted tool definitions—T17 rug pulls are out of scope.

ConfusedPilot [9] demonstrates confused deputy attacks in Copilot Workspace, aligning with MCPSecBench’s T02. However, it does not provide a CI-gatable testing framework.

AgentDojo [10] proposes adversarial benchmarks for autonomous agents, focusing on utility-safety trade-offs. Unlike `mcpsec`, it requires human-in-the-loop evaluation.

6.3 Continuous Security Testing

OWASP Dependency-Check and **Snyk** provide CI-integrated vulnerability scanning for dependencies, but do not address runtime exploit validation.

Semgrep and **CodeQL** enable static analysis of custom security rules, but cannot validate dynamic behaviors like rug pulls or sandbox escapes.

`mcpsec` bridges this gap by providing *dynamic, runtime-validated* threat testing in a CI-friendly package.

7 Conclusion

We presented `mcpsec`, a local CI gate for detecting server-side MCP security threats. By implementing 5 of MCPSecBench’s 17 vectors, we provide a practical, zero-dependency tool for shift-left security in LLM tool-calling infrastructure. Our dual-mode testing architecture ensures that each threat is both exploitable (validating the attack surface) and mitigable (validating the defense).

While `mcpsec` is explicitly *not* a full reproduction of MCPSecBench’s platform evaluations, it fills a critical gap: developers now have a `pip`-installable, 1.58-second CI check that validates basic server-side security properties against a well-defined threat taxonomy.

All code, tests, and gate outputs are available at <https://github.com/abinesha312/mcpsec> under the MIT license. We encourage the community to extend coverage to additional threat vectors and integrate `mcpsec` into production CI/CD pipelines.

Acknowledgments

This work was developed as a practical complement to MCPSecBench. We thank the authors of [1] for their systematic threat taxonomy, which enabled focused subset selection. We also acknowledge the broader AI safety and security community for ongoing efforts to secure LLM-agent ecosystems.

References

- [1] Yang, R., Wu, J., Chen, K. *MCPSecBench: A Systematic Security Benchmark and Playground for Testing Model Context Protocols*. arXiv preprint arXiv:2508.13220, 2025. <https://arxiv.org/abs/2508.13220>
- [2] Perez, F., Ribeiro, I., et al. *Ignore Previous Prompt: Attack Techniques For Language Models*. arXiv preprint arXiv:2211.09527, 2022.
- [3] Wei, A., Haghtalab, N., Steinhardt, J. *Jailbroken: How Does LLM Safety Training Fail?* NeurIPS, 2023.
- [4] Carlini, N., Tramer, F., et al. *Extracting Training Data from Large Language Models*. USENIX Security Symposium, 2021.
- [5] Zhu, K., Wang, J., et al. *PromptBench: Towards Evaluating the Robustness of Large Language Models on Adversarial Prompts*. arXiv preprint arXiv:2306.04528, 2023.
- [6] Sun, L., Huang, Y., et al. *TrustLLM: Trustworthiness in Large Language Models*. arXiv preprint arXiv:2401.05561, 2024.
- [7] Mazeika, M., Phan, L., et al. *HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal*. arXiv preprint arXiv:2402.04249, 2024.
- [8] Ruan, J., Chen, Y., et al. *Identifying the Risks of LM Agents with an LM-Emulated Sandbox*. arXiv preprint arXiv:2309.15817, 2023.
- [9] Zhang, Y., Liu, Y., et al. *Confused Deputy Attacks in AI-Powered Development Tools*. IEEE Security & Privacy, 2024.
- [10] De Benedetti, E., Kapoor, A., et al. *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. arXiv preprint arXiv:2406.13352, 2024.